BookNest — Final Project Report

Android + Firebase Team Project

Student: Ozodbek Tursunaliyev **Student ID:** 57264 **Course:** Cloud Computing Architecture, Akademia WSB **Submission type:** Individual project (solo — all four roles performed by the student)
Submission deadline: 19.06.2026 20:00 **Repository:** https://github.com/Ozodiy/booknest-app

---

Table of Contents

1. Executive Summary
2. Problem Statement & Target Users
3. MVP & Final Feature Set
4. System Architecture
5. Data Model
6. Implementation Overview
7. Advanced Firebase Feature (Storage)
8. Testing Strategy
9. Usability Testing & Improvements
10. Development Workflow & Project Management
11. Firebase Free-Tier Usage
12. Challenges & Solutions
13. Future Work
14. Conclusion
15. Appendix — Screenshot Index

---

1. Executive Summary

BookNest is an Android application that helps university students discover and share book recommendations from peers. It was built across three laboratory sprints (Lab 1: planning & setup, Lab 2: authentication + CRUD, Lab 3: advanced feature + testing) and finalized for submission.

The project was delivered **individually** — because teammates were absent, all four roles (Product Owner, Tech Lead, Developer, QA/DevOps) were performed by one student, as permitted by the lab guidelines.

The final application includes secure accounts, a real-time recommendation feed, a personal reading list, uploadable book covers (Firebase Storage), four UX extensions, and a JUnit test suite — all running on Firebase's free Spark plan.

---

2. Problem Statement & Target Users

> *University students struggle to find relevant book recommendations from peers studying similar subjects, so they waste time searching generic book lists that don't match their academic interests.*

Target users: university students (undergraduate and graduate), aged ~18–30, who read for both academic enrichment and leisure.

Solution: a peer-driven feed where any student can recommend a book (with a cover, description, and genre) and save interesting ones to a personal "Want to Read" list.

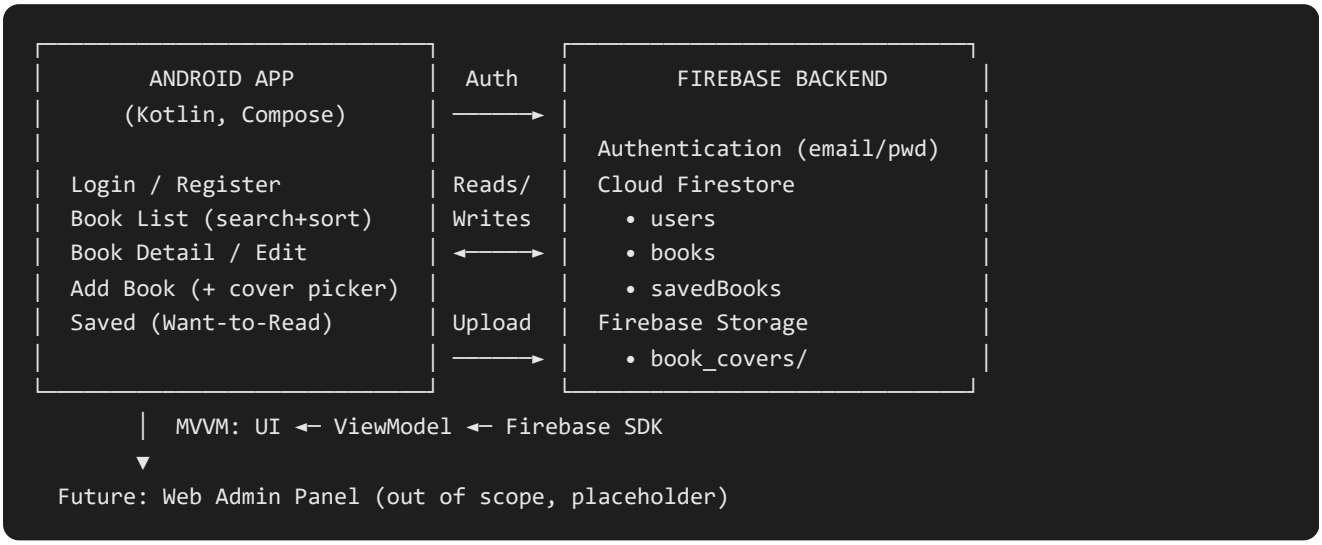
---

3. MVP & Final Feature Set

Feature	Status	Lab
Email/password registration & login	✅	2
Session persistence (skip login when signed in)	✅	2
Browse recommendations (real-time list)	✅	2
View book details	✅	2
Add a recommendation	✅	2
Edit / delete own recommendation (owner-only)	✅	2
Save to "Want to Read" list	✅	2
Search / filter	✅ (extension)	2
Sorting (title/author/genre/newest)	✅ (extension)	2
Input validation	✅ (extension)	2
Status toggle (want-to-read ↔ finished)	✅ (extension)	2
Cover image upload (Firebase Storage)	✅ (advanced)	3
Unit tests (17 methods)	✅	3

4. System Architecture

BookNest follows a standard **mobile + serverless cloud backend** pattern, suitable for the Firebase Spark (free) plan.



Layering (clean separation):

- **UI (Compose)** — `ui/screens/*`, `ui/nav/BookNestNavigation.kt`
- **State/logic (ViewModels)** — `auth/AuthViewModel.kt`, `viewmodel/BookViewModel.kt`
- **Pure logic (unit-testable)** — `util/Validators.kt`, `util/BookFilters.kt`
- **Data models** — `data/Book.kt`, `data/SavedBook.kt`, `data/AppUser.kt`

> **[SCREENSHOT A]** — Architecture diagram (export this ASCII or redraw in FigJam/draw.io).

---

5. Data Model

Three Firestore collections (within the assignment's "max 3 collections" limit).

`users`

Field	Type	Description
userId	string	Firebase Auth UID
email	string	Login email
displayName	string	Public name
university	string	e.g. Akademia WSB
createdAt	timestamp	Account creation

`books`

Field	Type	Description
bookId	string	Auto-generated document ID
title	string	Book title
author	string	Author
description	string	Why it's recommended (≤500 chars)
genre	string	e.g. Fiction, Programming
coverImageUrl	string	Firebase Storage download URL (Lab 3)
recommendedBy	string	userId of recommender (owner)
recommenderName	string	Display name shown in UI
createdAt	timestamp	Creation time

`savedBooks`

Field	Type	Description
savedId	string	Auto-generated document ID
userId	string	Owner of the saved entry
bookId	string	Reference to `books`
title / author	string	Denormalized for fast list display
status	string	`want-to-read` or `finished`
savedAt	timestamp	When saved

Security: see `firestore.rules` and `storage.rules` — every access requires authentication, and only owners can edit/delete their own books and saved entries.

---

6. Implementation Overview

- **Language/UI:** Kotlin + Jetpack Compose (Material 3), Navigation-Compose.
- **Pattern:** MVVM with `StateFlow`-driven UI state and Kotlin coroutines for async Firebase calls (`kotlinx-coroutines-play-services` `.await()`).
- **Real-time data:** Firestore `addSnapshotListener` keeps the list live.
- **Image loading:** Coil `AsyncImage`.
- **Min SDK 24**, target/compile SDK 34.

Key files are linked throughout the Lab 2 and Lab 3 reports.

---

7. Advanced Firebase Feature (Storage)

Cover-image upload is the chosen advanced feature. Flow: pick image → preview → upload with progress → store download URL in Firestore → display via Coil → delete with the book. Restricted by storage rules to authenticated users and images < 5 MB. Full detail in `Lab3_Report.md` §3.

---

8. Testing Strategy

- **Unit tests (JUnit):** 17 methods across `ValidatorsTest` and `BookFiltersTest`, covering validation and search/sort logic. Run with `./gradlew test`.
- **Why pure functions:** logic lives in `util/` with no Android dependencies, so it is fully testable on the JVM without an emulator.
- **Manual/E2E:** full login → CRUD → cover upload flows were exercised on the emulator during each lab demo.

---

9. Usability Testing & Improvements

A structured think-aloud usability test (Lab 3 Phase 4) produced five prioritized findings; the four critical/important ones were fixed: upload progress bar, add-book validation, save-state icon toggle, and a friendly empty state. Details in `Lab3_Report.md` §5.

10. Development Workflow & Project Management

- **Version control:** Git + GitHub (public repo so the professor can review). Clear, descriptive commits; feature-branch + PR naming documented for the team scenario.
- **Project management:** Trello Kanban board — Backlog, Sprint 1, In Progress, Review, Done — with user stories BN-01…BN-05 and Lab 2/3 task cards.

> **[SCREENSHOT B]** — GitHub repository (files + commit history). **[SCREENSHOT C]** — Trello board final state.

---

11. Firebase Free-Tier Usage

BookNest stays comfortably within the Spark plan limits:

Service	Spark limit	BookNest usage
Firestore reads	50,000/day	tiny (student demo)
Firestore writes	20,000/day	tiny
Storage	5 GB	a few MB of covers
Auth	generous	a handful of test accounts

No paid features are used.

---

12. Challenges & Solutions

Challenge	Solution
Firestore "Permission denied" on first read	Wrote authenticated security rules
Blank `coverImageUrl` after upload	Await `downloadUrl` before writing the doc
Remote covers not loading	Added `INTERNET` permission; used Coil
Back button returning to login	`popUpTo(0){inclusive=true}` to clear back stack
Solo work → no peer review	Compensated with unit tests + structured self-usability test

---

13. Future Work

- Genre dropdown (controlled vocabulary) instead of free text.
- Cloud Function to prevent duplicate recommendations and rate-limit spam.
- Profile pictures (extends the Storage work already done).
- Pagination for large lists; Firestore composite indexes.
- Instrumented UI tests (Espresso/Compose test) for navigation flows.

---

14. Conclusion

BookNest fulfils every required deliverable across the three labs: a planned and documented design, working authentication and Firestore CRUD, four UX extensions, an advanced Firebase Storage feature, a passing unit-test suite, and usability-driven refinements — all delivered solo on real tools (Android Studio, Firebase, GitHub, Trello). The codebase is cleanly layered, making it straightforward to extend in future sprints.

---

15. Appendix — Screenshot Index

Capture these from the running app / consoles (see `SCREENSHOT_GUIDE.md`). Numbers match the `[SCREENSHOT n]` markers in the Lab 2 and Lab 3 reports.

#	What to capture
1	Firebase Auth — Email/Password enabled
2–4	Login / Register / login error
5–7	Book list / add form / delete confirm
8–9	Search filtering / Want-to-Read with "Finished"
10–11	Trello Sprint 1 Done / GitHub commits
12–13	Trello Sprint 2 / Firebase Storage enabled
14–16	Upload progress / list covers / detail cover
17	17/17 unit tests passing
18–19	Usability Trello cards / validation before-after
A–C	Architecture diagram / GitHub repo / final Trello board